스트리밍 알고리즘 4종 구현과 정확도·메모리·시간 트레이드오프 분석

윤승희 · 학번 2243175 · 2026년 11월 6일
소스코드: https://github.com/realp0tato/streaming-algorithms-movielens

핵심 결과

- **메모리 절감:** HyperLogLog는 19.2 KB로 고유 쌍 1,000,209개를 RMSE 0.72%로 추정했습니다. 정확한 Python set(83.12 MB)의 약 4,335분의 1입니다.
- **파라미터는 클수록 좋지 않습니다:** Bloom Filter는 m/n=4에서 k를 3에서 14로 늘리면 FPR이 14.7%에서 65.0%로 나빠졌고, Count-Min은 같은 메모리에서 depth에 최적점이 있었습니다.
- **시간은 오히려 손해입니다:** 순수 Python 구현에서는 정확한 자료구조가 스케치보다 빨랐습니다. 이번 실험에서 스케치의 이점은 메모리입니다.
- **결론:** 가장 실용적인 알고리즘은 HyperLogLog입니다. 실제 서비스 로그라면 고유 사용자 집계에 HLL을 먼저 적용하고, 인기 항목 집계에 Count-Min을 더하겠습니다.

1. 데이터셋 설명

MovieLens 1M(GroupLens)을 사용했습니다. 사용자::영화::평점::시각 형식의 평점 이벤트 1,000,209건으로, 과제 조건인 100,000건 이상의 텍스트 로그를 충족합니다. 파일은 압축을 풀지 않고 한 줄씩 읽었으며, Python 힙 최고 사용량은 **90.3 KB**로 전체를 적재할 때의 299.89 MB보다 훨씬 작습니다. 라이선스가 재배포를 금지하므로 데이터는 저장소에 넣지 않고, 실행 시 내려받아 SHA-256으로 확인합니다.

항목	값	항목	값
레코드 수	1,000,209	평균 평점	3.582
고유 사용자	6,040	사용자당 평점 수	최소 20 / 중앙값 96 / 최대 2,314
고유 영화	3,706	영화당 평점 수	최소 1 / 중앙값 123.5 / 최대 3,428
고유 (사용자, 영화) 쌍	1,000,209	상위 10% 영화의 평점 비중	44.4%

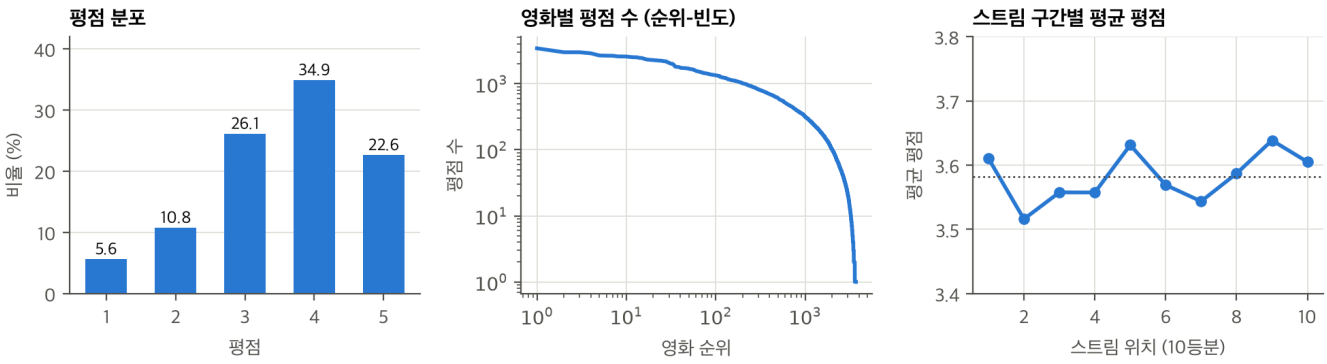


그림 1. 영화 빈도는 긴 꼬리 분포입니다. 평점 수 10 이하인 영화가 473개입니다. 구간별 평균은 사용자 구성에 따라 3.52~3.64 범위에서 변합니다.

실험에 영향을 주는 특성이 두 가지 있습니다. 첫째, 파일이 **사용자 ID 순으로 정렬**되어 있어 앞의 1,000건에는 사용자가 10명뿐입니다. 스트림 앞부분만 쓰면 표본이 치우치므로 Reservoir Sampling의 효과를 검증하기 좋습니다. 둘째, 시각 순서가

아니므로(인접 레코드 시각 역전 488,107회) 실시간 로그의 도착 순서를 그대로 재현하지는 않습니다.

2. 선택한 알고리즘 개요

과제는 2개 이상을 요구하지만, 스트리밍 문제 유형별 비교를 위해 4종을 모두 구현했습니다.

알고리즘	질문	자료구조	오차의 방향과 이론값	스트림 키
Bloom Filter	이 원소를 본 적이 있는가?	m비트 배열, 해시 k개	거짓 음성 없음. FPR ≈ $(1-e^{-kn/m})^k$, 최적 k ≈ 0.693·m/n	사용자:영화 쌍
Count-Min Sketch	이 항목이 몇 번 나왔는가?	d×w 카운터	과대추정만 발생. 확률 $1-e^{-d}$ 이상으로 오차 ≤ (e/w)·N	영화 ID
HyperLogLog	서로 다른 원소가 몇 개인가?	2^p개 레지스터	상대 표준오차 ≈ $1.04/\sqrt{(2^p)}$	쌍, 사용자, 영화
Reservoir Sampling	균등 표본은 무엇인가?	크기 k 배열	모든 원소의 포함 확률이 k/n. 추정 오차는 약 $1/\sqrt{k}$로 감소	레코드

3. 구현 방식

- **직접 구현:** 네 알고리즘의 핵심 로직은 라이브러리 없이 표준 라이브러리만으로 `sketches.py`에 구현했습니다. numpy와 scipy는 결과 평가에만 씁니다.
- **해시:** 모두 `blake2b`에 시드를 salt로 넣어 사용합니다. Python 내장 `hash()`는 실행마다 값이 달라져 재현성을 해치기 때문입니다.
- **Bloom Filter:** 이중 해싱($g_i = h_1 + i \cdot h_2$)으로 한 번의 해시 계산에서 k개 위치를 만듭니다. 조회는 0인 비트를 만나면 바로 끝납니다.
- **Count-Min:** 이중 해싱을 쓰면 두 원소가 전 행에서 함께 충돌할 확률이 커지므로, 해시 출력을 행 수만큼 나눠 행별로 독립적인 값을 씁니다. 확장으로 보수적 갱신(Conservative Update)도 구현했습니다.
- **HyperLogLog:** 상위 p비트로 레지스터를 고르고 나머지 비트의 첫 1 위치를 기록합니다. 소규모 구간은 선형 계수로 보정합니다. 레지스터는 이론상 6비트면 충분하지만 바이트 단위로 저장했습니다.
- **Reservoir:** 원소마다 난수를 뽑는 Algorithm R과, 다음 교체 위치까지 건너뛰는 확장 기법 Algorithm L을 구현했습니다.
- **Ground Truth:** 같은 스트림을 한 번 읽어 `set`과 `Counter`로 정확한 값을 계산했습니다. Bloom Filter의 음성 질의는 실제 사용자×영화 조합 중 기록이 없는 쌍 1,000,000개를 시드별로 뽑았습니다.

```
# HyperLogLog: 레지스터 선택과 첫 1비트 위치 기록
def add(self, item):
    h = int.from_bytes(blake2b(item.encode(), digest_size=8, salt=self.salt).digest(), "little")
    i = h >> self.q                                  # 상위 p비트 = 레지스터 번호
    rank = self.q - (h & self.mask).bit_length() + 1  # 나머지 비트의 첫 1 위치
    if rank > self.reg[i]:
        self.reg[i] = rank

# Count-Min: 갱신은 행마다 +1, 조회는 행들의 최솟값
def query(self, item):
    return min(self.table[c] for c in self._cells(item))
```

정확도 실험에서는 알고리즘의 구조적 성질을 함께 확인했습니다. Bloom Filter 전 시행의 거짓 음성은 0건이고, Count-Min은 모든 영화에서 과소추정이 없었으며(코드의 assert로 검사), Reservoir는 표본 크기와 중복 없음을 매번 검사합니다.

4. 실험 환경

항목	내용
하드웨어	Apple M1, 8코어, RAM 8 GB
소프트웨어	macOS 26.6, CPython 3.12.13, numpy 2.5.4, pandas 3.1.1
정확도 측정	시드별 반복(Bloom 3, Count-Min 5, HLL 15, Reservoir 30회). 6개 프로세스 병렬 실행. 재실행 시 결과 파일이 동일함을 확인
시간 측정	단일 프로세스, 파일에서 직접 스트리밍. 전체 설정을 매 반복 섞어 7회 측정한 중앙값
메모리 측정	tracemalloc의 Python 힙 사용량. 처리 시간을 늘리므로 시간 측정과 별도로 1회 실행

5. 파라미터 설정

알고리즘	비교 파라미터	시행 수
Bloom Filter	m/n ∈ {2, 4, 6, 8, 10, 12, 16, 20}, k ∈ {1, 2, 3, 4, 5, 7, 10, 14}	64조합 × 3 시드
Count-Min	w ∈ {64, 128, ⋯, 32768}, d ∈ {1, 2, 4, 8, 16}. 보수적 갱신은 d ∈ {2, 4}	70조합 × 5 시드
HyperLogLog	p ∈ {4, ⋯, 16} (레지스터 16~65,536개), 대상 3종	39조합 × 15 시드
Reservoir	k ∈ {100, 500, 1K, 5K, 10K, 50K, 100K}. R, L, 전체 데이터 무작위 표본(기준), 앞에서 k개(대조군)	시드 30개, 대조군은 1회

6. 정확도 비교 결과

6-1. Bloom Filter

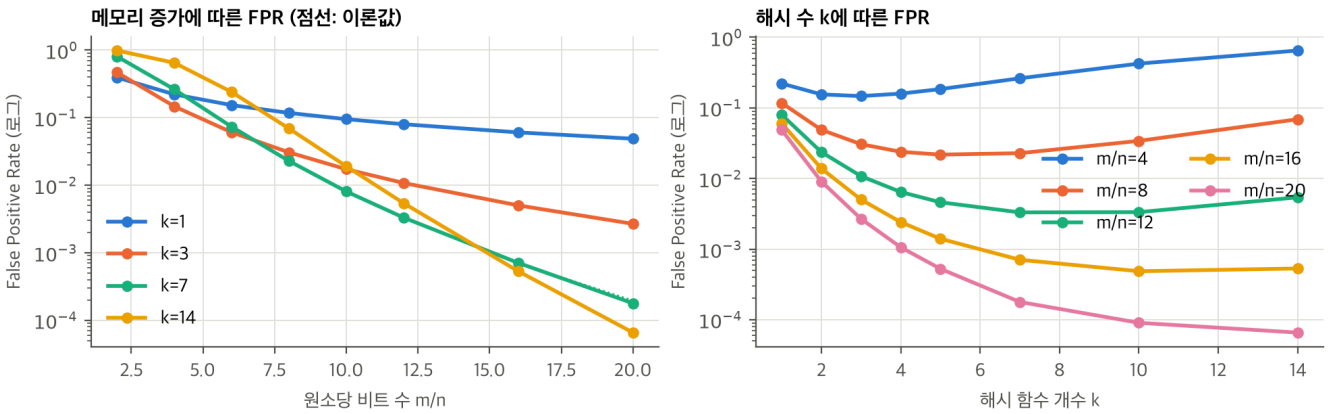


그림 2. 오른쪽 그림처럼 k에 따라 FPR이 U자로 변하며, 메모리가 클수록 최적 k도 커집니다.

m/n	크기	k=1	k=2	k=3	k=4	k=5	k=7	k=10	k=14
2	250.1 KB	**0.39369**	0.39993	0.46865	0.55852	0.65115	0.80651	0.93413	0.98734
4	500.1 KB	0.22143	0.15528	**0.14702**	0.15941	0.18474	0.26269	0.42418	0.64997
6	750.2 KB	0.15347	0.08047	0.06069	**0.05578**	0.05757	0.07334	0.12352	0.23995
8	1.00 MB	0.11751	0.04892	0.03061	0.02389	**0.02178**	0.02285	0.03412	0.06907
10	1.25 MB	0.09500	0.03289	0.01725	0.01172	0.00940	**0.00807**	0.01013	0.01906
12	1.50 MB	0.08000	0.02367	0.01078	0.00644	0.00460	**0.00331**	0.00334	0.00541
16	2.00 MB	0.06050	0.01384	0.00502	0.00241	0.00140	0.00071	**0.00049**	0.00053
20	2.50 MB	0.04878	0.00907	0.00268	0.00106	0.00053	0.00018	0.00009	**0.00007**

값은 FPR, 파란 숫자는 행별 최솟값입니다.

측정 FPR은 이론값과 잘 맞습니다. 거짓 양성이 300건 이상인 조합에서 측정값/이론값은 0.910~1.034입니다. m/n=10의 최적 k는 이론값 6.9에 가까운 k=7(FPR 0.807%)입니다. 메모리를 2배로 늘리면(m/n=10→20) FPR은 약 123분의 1이 되어, 메모리 대비 오차 감소가 네 알고리즘 중 가장 빠릅니다.

6-2. Count-Min Sketch

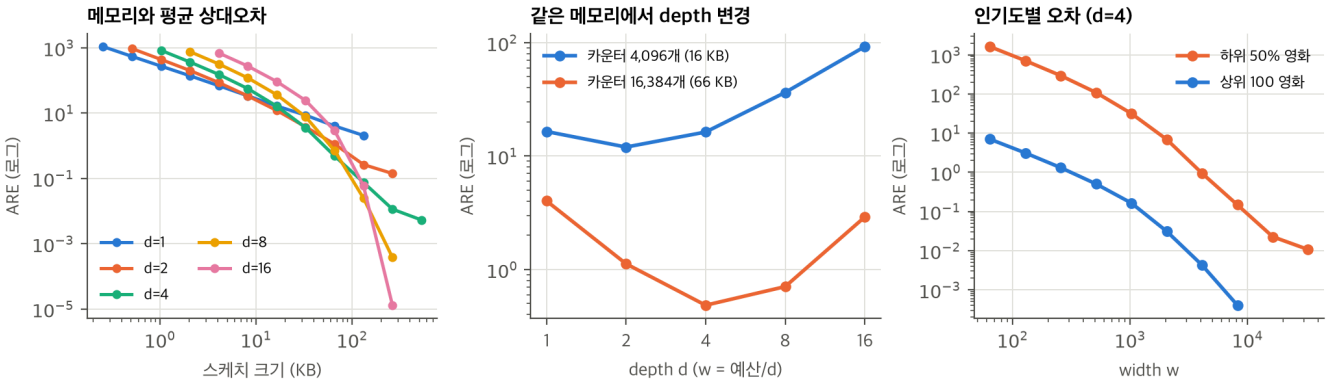


그림 3. 왼쪽: 메모리와 오차. 가운데: 같은 메모리에서 depth를 바꾼 결과. 오른쪽: 인기 영화와 비인기 영화의 오차 차이.

w	d=1	d=2	d=4	d=8	d=16	d=4 크기
64	1078.713	937.241	830.189	756.508	685.759	1.0 KB
128	540.706	436.818	364.346	317.256	273.676	2.0 KB
256	272.862	200.527	151.341	118.973	92.246	4.1 KB
512	138.286	85.436	56.005	36.408	24.174	8.2 KB
1,024	68.468	33.400	16.296	7.677	2.902	16.4 KB
2,048	33.581	12.012	3.555	0.707	0.059	32.8 KB
4,096	16.465	3.770	0.484	0.025	0.000	65.5 KB
8,192	8.575	1.123	0.075	0.000	0.000	131.1 KB
16,384	4.024	0.265	0.011	0.000	0.000	262.1 KB
32,768	2.064	0.141	0.005	0.000	0.000	524.3 KB

값은 전체 영화에 대한 평균 상대오차(ARE)입니다.

- **오차는 비인기 항목에 몰립니다.** w=2048, d=4에서 상위 100 영화의 ARE는 3.1%지만 하위 50%는 6.93, 즉 실제 값의 약 8배로 추정합니다. 충돌로 더해지는 절대 오차가 항목 크기와 무관하기 때문입니다. Count-Min은 인기 항목 탐지에는 적합하지만 드문 항목의 빈도 추정에는 부적합합니다.
- **이론 보장은 지켜졌습니다.** 모든 (w, d)에서 오차가 (e/w)·N을 넘는 영화의 비율이 $\delta = e^{-d}$ 이하였습니다. 다만 이 경계는 실측 오차보다 느슨합니다.
- **보수적 갱신:** w=2048, d=4에서 MAE가 54.5→19.8, ARE가 3.56→2.63로 줄었고, 메모리는 같지만 갱신 시간이 1.72→2.49 µs로 늘었습니다.

6-3. HyperLogLog

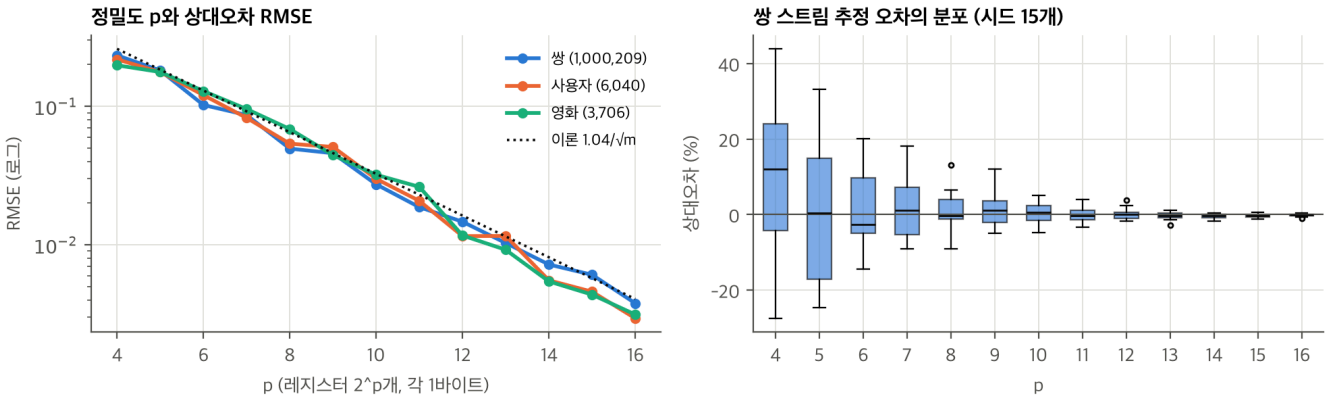


그림 4. 측정 오차가 이론 곡선을 따릅니다. p가 작으면 시드에 따른 편차가 큽니다.

p	레지스터	크기	이론 오차	쌍	사용자	영화
4	16	0.0 KB	26.00%	23.35%	21.83%	19.77%
6	64	0.1 KB	13.00%	10.22%	12.04%	12.88%
8	256	0.3 KB	6.50%	4.95%	5.37%	6.85%
10	1,024	1.0 KB	3.25%	2.73%	2.99%	3.20%
12	4,096	4.1 KB	1.62%	1.47%	1.16%	1.17%
14	16,384	16.4 KB	0.81%	0.72%	0.55%	0.54%
16	65,536	65.5 KB	0.41%	0.38%	0.29%	0.31%

값은 상대오차 RMSE이며 시드 15개 기준입니다.

로그-로그 기울기는 -0.49로 이론값 −0.5와 비슷합니다. 메모리를 4배로 늘려야 오차가 절반이 됩니다. 쌍, 사용자, 영화의 실제 값이 최대 약 270배 차이 나도 오차 수준이 비슷해, 메모리가 카디널리티와 거의 무관하다는 점도 확인했습니다. p=4의 쌍 스트림에서는 평균 편향이 +8.8%로 커서, 이 정도 크기는 실사용에 부적합합니다.

6-4. Reservoir Sampling

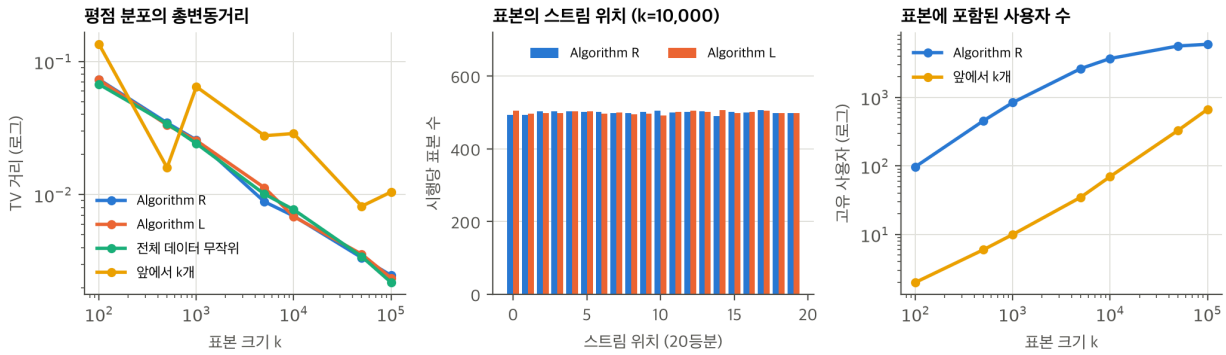


그림 5. 왼쪽: 평점 분포 거리. 가운데: 표본이 스트림 전체에 균등하게 분포합니다. 오른쪽: 앞에서 k개를 고르면 사용자 다양성이 크게 떨어집니다.

k	TV: R	TV: L	TV: 전체 무작위	TV: 앞 k개	평균 오차: R	평균 오차: 앞 k개	사용자: R	사용자: 앞 k개
100	0.0734	0.0729	0.0675	0.1348	0.0829	0.3684	98	2
1,000	0.0257	0.0255	0.0242	0.0647	0.0268	0.1954	843	10
10,000	0.0069	0.0068	0.0077	0.0288	0.0071	0.0523	3,696	70
100,000	0.0025	0.0024	0.0022	0.0105	0.0030	0.0286	5,958	669

TV는 표본과 전체 평점 분포 사이의 총변동거리이고, 평균 오차는 평균 평점의 절대 오차입니다.

R과 L은 Ground Truth에 해당하는 전체 데이터 무작위 표본과 같은 수준의 오차를 보입니다. 위치 균등성도 검정했습니다. 시행별 KS 검정에서 p < .05 비율은 4.5%로 우연 기대치 5%와 비슷하고, k=10,000 표본을 합친 20구간 카이제곱 검정의 p값은 R 0.55, L 0.49입니다. 균등하지 않다는 증거는 없습니다. 반면 앞에서 k개를 고르면 k=1,000일 때 사용자가 10명뿐입니다. 이 대조군은 시드가 없어 1회 값이며, k=500의 낮은 평균 오차(0.0104)는 우연히 맞은 경우입니다. 로그-로그 기울기는 -0.50로 이론값 −0.5에 가깝습니다.

7. 메모리 사용량 비교 결과

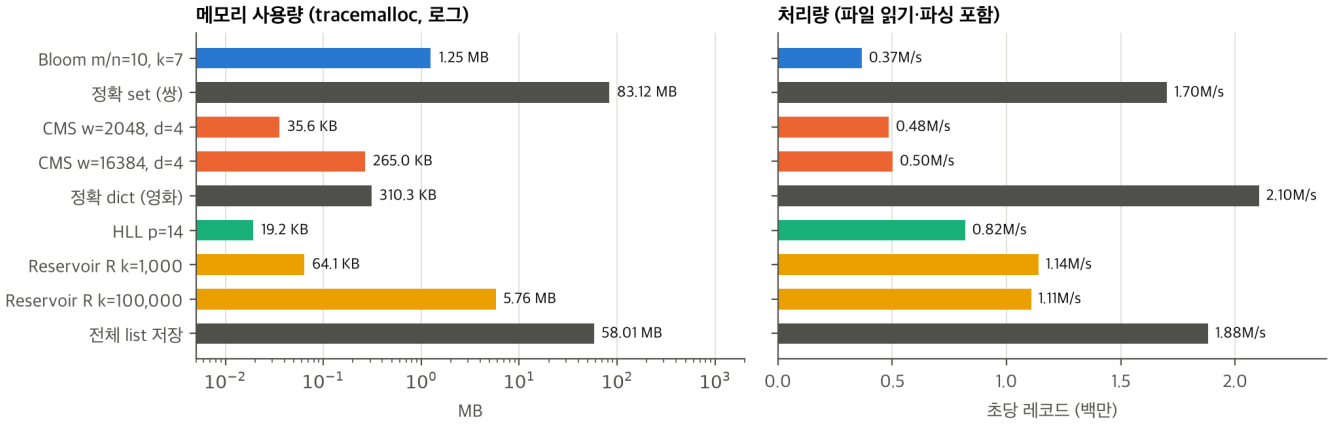


그림 6. 대표 설정의 메모리(로그 축)와 처리량. 회색은 정확한 자료구조입니다.

구분	설정	처리 후 유지	최고(스트림 제외)	자료구조 크기
Bloom Filter	m/n=4, k=3	502.8 KB	502.6 KB	500.1 KB
Bloom Filter	m/n=10, k=1	1.25 MB	1.25 MB	1.25 MB
Bloom Filter	m/n=10, k=4	1.25 MB	1.25 MB	1.25 MB
Bloom Filter	m/n=10, k=7	1.25 MB	1.25 MB	1.25 MB
Bloom Filter	m/n=10, k=14	1.25 MB	1.25 MB	1.25 MB
Bloom Filter	m/n=20, k=14	2.50 MB	2.50 MB	2.50 MB
정확(set)	pair set	83.12 MB	83.09 MB	-
Count-Min	w=2048, d=1	11.0 KB	8.6 KB	8.2 KB
Count-Min	w=2048, d=4	35.6 KB	33.2 KB	32.8 KB
Count-Min	w=2048, d=16	134.1 KB	131.6 KB	131.1 KB
Count-Min	w=16384, d=4	265.0 KB	262.5 KB	262.1 KB
Count-Min	w=2048, d=4, CU	35.7 KB	33.2 KB	32.8 KB
정확(dict)	movie dict	310.3 KB	307.1 KB	-
HyperLogLog	p=6	2.8 KB	2.6 KB	0.1 KB
HyperLogLog	p=10	3.8 KB	3.6 KB	1.0 KB
HyperLogLog	p=14	19.2 KB	18.9 KB	16.4 KB
HyperLogLog	p=16	68.3 KB	68.1 KB	65.5 KB
Reservoir	R, k=1000	64.1 KB	63.7 KB	-
Reservoir	R, k=100000	5.76 MB	5.76 MB	-

구분	설정	처리 후 유지	최고(스트림 제외)	자료구조 크기
Reservoir	L, k=1000	64.2 KB	63.8 KB	-
Reservoir	L, k=100000	5.76 MB	5.76 MB	-
정확(list)	전체 저장	58.01 MB	57.99 MB	-

유지와 최고 메모리는 tracemalloc 값입니다. 자료구조 크기는 비트 배열·카운터·레지스터의 바이트 수이며, 표본이나 Python 객체는 해당하지 않아 "-"로 표시했습니다.

정확한 자료구조와의 비교는 데이터 특성에 좌우됩니다. 측정값은 Python 객체 오버헤드를 포함하므로 set의 83.12 MB은 다소 불리한 비교입니다. 사용자×영화 조합이 유한한 이 데이터에서는 조합마다 1비트씩 쓰는 정확한 비트맵이 약 2.98 MB이어서 (계산값), m/n=20의 Bloom Filter(2.50 MB)와 큰 차이가 없습니다. 영화 빈도도 키가 3,706개뿐이라 정확한 dict가 310.3 KB로 작고, 오차가 작은 w=16384, d=4(265.0 KB)와 비슷합니다. 즉 키 공간이 작으면 스케치의 이점이 줄고, URL이나 IP처럼 키 공간이 크거나 미리 알 수 없을 때 이점이 커집니다.

8. 처리 시간 비교 결과

구분	설정	전체(초)	최소~최대	처리량	순처리(µs/건)
Bloom Filter	m/n=4, k=3	1.84	1.62~4.10	542K/s	1.41
Bloom Filter	m/n=10, k=1	1.45	1.26~2.76	688K/s	1.02
Bloom Filter	m/n=10, k=4	2.25	1.85~6.78	445K/s	1.81
Bloom Filter	m/n=10, k=7	2.72	2.34~5.16	367K/s	2.29
Bloom Filter	m/n=10, k=14	4.68	3.51~10.01	214K/s	4.24
Bloom Filter	m/n=20, k=14	4.07	3.49~7.23	246K/s	3.63
정확(set)	pair set	0.59	0.49~2.19	1,702K/s	0.15
Count-Min	w=2048, d=1	1.51	1.31~3.11	662K/s	1.17
Count-Min	w=2048, d=4	2.06	1.85~4.57	485K/s	1.72
Count-Min	w=2048, d=16	4.55	3.83~7.31	220K/s	4.20
Count-Min	w=16384, d=4	1.99	1.83~6.87	502K/s	1.65
Count-Min	w=2048, d=4, CU	2.83	2.34~3.62	353K/s	2.49
정확(dict)	movie dict	0.48	0.41~0.65	2,104K/s	0.13
HyperLogLog	p=6	1.18	1.10~2.34	844K/s	0.75
HyperLogLog	p=10	1.31	1.06~2.02	765K/s	0.87
HyperLogLog	p=14	1.22	1.05~2.27	819K/s	0.78
HyperLogLog	p=16	1.18	1.09~1.70	849K/s	0.74
Reservoir	R, k=1000	0.88	0.76~2.11	1,140K/s	0.44
Reservoir	R, k=100000	0.90	0.79~2.76	1,108K/s	0.47
Reservoir	L, k=1000	0.57	0.46~0.59	1,742K/s	0.14
Reservoir	L, k=100000	0.87	0.70~3.29	1,151K/s	0.43
정확(list)	전체 저장	0.53	0.39~0.73	1,882K/s	0.09

전체 시간과 처리량은 파일 읽기와 파싱을 포함합니다. 순처리는 파싱만 수행한 기준 시간(쌍 0.44초, 영화 0.34초)을 뺀 값입니다.

조회 작업	초당 처리량
Bloom m/n=10, k=7, 양성	565K/s
Bloom m/n=10, k=7, 음성	917K/s
Bloom m/n=20, k=14, 양성	340K/s
Bloom m/n=20, k=14, 음성	905K/s
정확 set, 음성	14,549K/s
Count-Min w=2048, d=4	608K/s
HyperLogLog p=14 estimate()	964/s

- **순수 Python에서는 정확한 구조가 더 빠릅니다.** set은 건당 0.15 µs인데 Bloom Filter(k=7)는 2.29 µs입니다. 스케치는 해시를 Python에서 호출하고 위치마다 반복하지만, set은 C로 구현되어 있기 때문입니다. C나 Rust 구현이라면 격차가 달라질 수 있으나 이번에는 측정하지 않았습니다.
- **시간은 메모리 크기가 아니라 해시·갱신 횟수에 좌우됩니다.** Count-Min은 d=1→16에서 1.17→4.20 µs로 늘었지만, w=2048→16384에서는 1.72→1.65 µs로 거의 그대로입니다. Bloom Filter도 k=1→14에서 1.02→4.24 µs로 늘었습니다. HLL은 p에 관계없이 약 0.8 µs입니다.
- **Bloom Filter 조회:** k=14, m/n=20에서 음성 조회(905K/s)가 양성 조회(340K/s)보다 빠릅니다. 0인 비트를 만나면 바로 끝나기 때문입니다.
- **Algorithm L:** k=1,000에서 R보다 빠르지만(0.44→0.14 µs), k=100,000에서는 교체가 잦아 차이가 줄어듭니다(0.47→0.43 µs). 파일을 어차피 한 줄씩 읽어야 하므로 전체 처리량 향상은 더 작습니다.
- **측정 잡음:** 일부 설정은 최소~최대 폭이 넓습니다. m/n=10, k=14보다 m/n=20, k=14가 빠르게 나온 것처럼 수십 % 이내의 차이는 잡음일 수 있으니 큰 경향만 해석해야 합니다.

9. 알고리즘별 장단점 분석

알고리즘	설정	정확도	메모리	정확한 방식	절감	처리량
Bloom Filter	m/n=10, k=7	FPR 0.81%	1.25 MB	83.12 MB	66배	367K/s
Count-Min	w=2048, d=4	상위 100 ARE 3.1%, 전체 ARE 3.56	35.6 KB	310.3 KB	9배	485K/s
Count-Min	w=16384, d=4	상위 100 ARE 0.00%, 전체 ARE 0.011	265.0 KB	310.3 KB	1.2배	502K/s
HyperLogLog	p=14	RMSE 0.72%	19.2 KB	83.12 MB	4,335배	819K/s
Reservoir (R)	k=10,000 *	TV 0.0069, 평균 오차 0.0071	576.2 KB	58.01 MB	101배	1,140K/s

* Reservoir의 메모리는 k=100,000 측정값으로 환산했고 처리량은 k=1,000 측정값입니다. Count-Min은 키 공간이 작아 절감 폭이 작습니다.

알고리즘	장점	단점	적합한 곳
Bloom Filter	거짓 음성이 없음. 메모리 대비 오차 감소가 가장 빠름	원소를 꺼내거나 삭제할 수 없음. n을 미리 알아야 m, k를 정할 수 있고 n을 넘기면 FPR 급증	중복 제거, 캐시 앞단 조회 차단
Count-Min	과대추정만 발생하며 인기 항목에 정확함. 갱신 시간이 w와 무관	비인기 항목의 상대오차가 큼. 키 공간이 작으면 dict와 차이가 작음. 키 목록은 저장하지 않음	인기 항목 집계, 트래픽 모니터링
HyperLogLog	수십 KB로 약 1% 오차. 파라미터가 p 하나이고 오차 예측이 쉬움	고유 수만 알 수 있고 어떤 원소인지 모름. 소규모 p에서 편향	순방문자, 고유 IP 집계
Reservoir	원본 레코드가 남아 사후 분석이 자유로움. 구현이 간단하고 정렬된 스트림에도 편향 없음	메모리가 k에 비례하고 레코드 크기에도 좌우됨. 희귀 항목은 표본에 잘 잡히지 않음	로그 표본 저장, 디버깅, 분포 점검

10. 결론: 최종 분석 질문

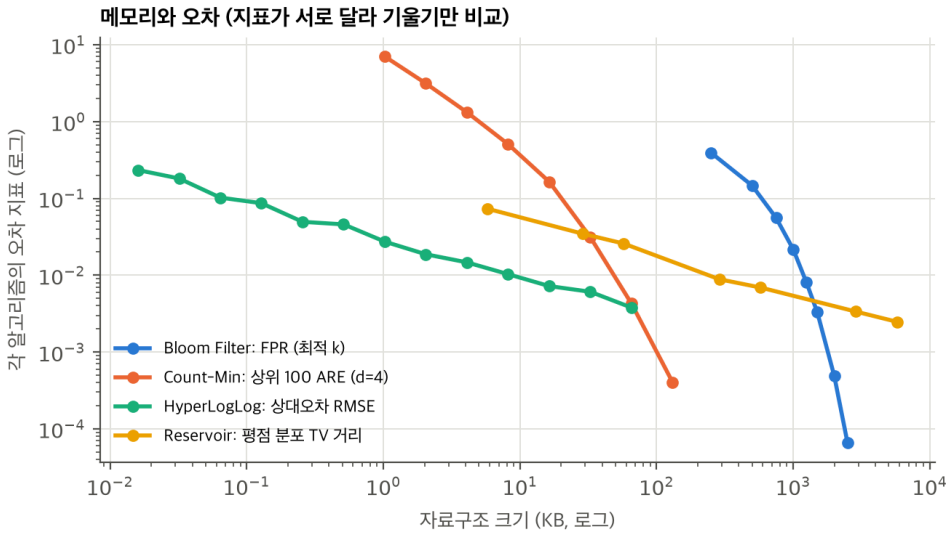


그림 7. 알고리즘별 정확도-메모리 곡선. 오차 지표가 달라 높이는 비교할 수 없고 기울기만 비교합니다.

Q1. 정확도와 메모리 사이에는 어떤 trade-off가 있었는가?

메모리를 늘리면 오차는 줄지만, 줄어드는 속도는 알고리즘마다 크게 달랐습니다.

- **Bloom Filter는 지수적입니다.** 최적 k 기준으로 메모리 2배(m/n 10→20)에 FPR이 0.807%→0.0066%로 약 123분의 1이 됩니다.
- **HyperLogLog와 Reservoir는 제곱근입니다.** 측정 기울기 -0.49, -0.50로 메모리를 4배 써야 오차가 절반입니다. 그래서 초반 절감은 크지만 높은 정밀도는 비쌉니다.
- **Count-Min은 충돌이 사라지는 구간에서 급격히 좋아집니다.** d=1에서는 w에 거의 반비례(기울기 -1.01)하지만, d=4는 w가 키 수 3,706 근처를 넘으면서 ARE가 16.3→0.075로 빠르게 떨어집니다.
- **이점의 크기는 데이터에 달려 있습니다.** HLL은 정확한 set의 약 4,335분의 1이지만, Count-Min w=16384, d=4는 dict의 약 1.2분의 1에 그칩니다. 스케치는 키 공간이 클수록 유리합니다.

Q2. 파라미터 증가가 항상 성능 향상으로 이어졌는가?

아닙니다. 파라미터를 키워도 정확도가 나빠지거나, 정확도 이득 없이 시간과 메모리만 늘어나는 경우가 있었습니다.

- **Bloom Filter의 k: 정확도가 나빠졌습니다.** m/n=4에서 k=3의 FPR은 14.7%이지만 k=14는 65.0%입니다. 비트가 채워져 판별력이 사라지기 때문입니다. 처리 시간도 k에 비례해 늘어납니다.
- **Count-Min의 d: 메모리가 고정되면 최적점이 있습니다.** 카운터 16,384개에서 d=1, 2, 4, 8, 16의 ARE는 4.02, 1.12, 0.48, 0.71, 2.90입니다. d를 너무 키우면 w가 줄어 오차가 다시 커지고, 시간은 계속 늘어납니다.
- **메모리만 늘리는 경우엔 수렴합니다.** d=4에서 w=16384→32768은 ARE가 0.011→0.005로 이미 작은 오차를 조금 더 줄일 뿐, 메모리는 2배입니다.
- **HLL의 p, Reservoir의 k:** 정확도는 대체로 개선되지만 제곱근 법칙으로 이득이 줄어듭니다. HLL은 p=14→16에서 메모리가 4배인데 RMSE는 0.72%→0.38%로 줄어드는 데 그칩니다.

Q3. 어떤 알고리즘이 가장 실용적이라고 판단되는가?

HyperLogLog입니다. 네 가지 중 메모리 대비 효과가 가장 크고 설정이 가장 쉽습니다.

19.2 KB로 1,000,209개의 고유 쌍을 RMSE 0.72%로 추정했고, 대상 크기가 최대 약 270배 달라도 오차가 비슷했습니다. 파라미터는 p 하나이며 1.04/√m으로 필요한 크기를 미리 계산할 수 있습니다. Bloom Filter는 n을 미리 알아야 하고, Count-Min은 드문 항목의 오차가 크며 이번 데이터에서는 dict 대비 이점도 작았습니다. 다만 이 판단은 "고유 개수"라는 질문에 한정됩니다. 포함 여부나 빈도를 물어야 한다면 HLL로는 답할 수 없습니다.

Q4. 실제 서비스 로그 분석에 적용한다면 어떤 알고리즘을 선택할 것인가?

HyperLogLog를 먼저 도입하고, 인기 항목 집계에 Count-Min Sketch를 더하겠습니다.

- **1순위 HyperLogLog (DAU, 페이지별 순방문자, 고유 IP):** 서비스 로그에서 가장 흔하고 정확한 집계 비용이 가장 큰 지표입니다. p=14 정도면 지표당 약 16 KB로 약 1% 오차를 기대할 수 있어, 지표가 많아도 부담이 작습니다.
- **2순위 Count-Min (인기 URL, 트래픽 급증 키):** 키 공간이 크고 관심 대상이 상위 항목이라 이번에 확인한 "상위 항목에 정확함"이 그대로 강점이 됩니다. 단, 스케치에는 키 목록이 없어 상위 항목을 뽑으려면 별도의 힙이 필요합니다.
- **보조 Reservoir (장애 조사용 표본):** 정렬되거나 치우친 로그에서도 편향 없는 원본 표본을 남길 수 있습니다.
- **필요할 때 Bloom Filter (이벤트 중복 제거):** 거짓 양성은 신규 이벤트를 중복으로 잘못 버리는 결과가 되므로, 데이터 손실을 허용할 수 있을 때만 쓰겠습니다.

운영에서는 Python 구현을 그대로 쓰지 않겠습니다. 이번 측정에서 처리량이 약 819K건/초로, 단일 프로세스 Python에서는 대규모 스트림을 감당하기 어렵습니다. Redis의 HLL처럼 검증된 네이티브 구현을 쓰는 편이 낫습니다. 시간별 HLL을 합치는 병합 연산도 실무에서 중요하지만 이번 과제에서는 구현하거나 측정하지 않았습니다.

한계

- MovieLens 1M은 키 공간이 작고 쌍이 모두 고유합니다(같은 쌍의 반복이 없음). 그래서 Count-Min의 메모리 이점을 작게, Python set과의 비교에서 Bloom Filter·HLL의 이점을 크게 보여 줄 수 있습니다.
- 파일이 시간순이 아니어서 실제 로그의 시간적 변화나 트래픽 급증은 재현하지 못했습니다.
- Bloom Filter의 FPR이 매우 낮은 조합은 거짓 양성이 수십~수백 건뿐이라 상대적으로 불확실합니다.
- 시간은 M1 한 대의 순수 Python 값이므로, 절대값보다 설정 간 경향을 봐야 합니다.

재현 방법

소스코드: https://github.com/realp0tato/streaming-algorithms-movielens

```
python3.12 -m venv .venv && .venv/bin/pip install -r requirements.txt
.venv/bin/python sketches.py          # 알고리즘 자체 검증
.venv/bin/python run_experiments.py   # 데이터 다운로드와 전체 실험 (M1 기준 약 20분)
.venv/bin/python build_report.py <github-url>   # 그림과 이 PDF 생성
```